

프로그래밍의 새로운 병목

김병권, KAIST AI컴퓨팅학과

본 에세이는 빠르게 발전하는 인공지능의 시대에 프로그래밍의 새로운 병목은 의도의 전달에 있을 것이라 예측하며, 미래의 프로그래밍은 의도의 전달을 최대한 효율적이고 직접적으로 할 수 있는 방식으로 진화할 것이라 주장한다.

최근 수년간 거대 언어 모델(Large Language Model, LLM)의 놀라운 발전은 여러 분야에 충격을 주었지만 그 중에서도 실제 업무의 과정 그리고 방식이 가장 많이 변한 작업 중 하나는 코딩이다. 새내기 대학생부터 수십 년 경력의 저명한 프로그래머까지[4], 코딩을 하는 사람이라면 거의 누구나 소위 바이브 코딩(vibe coding), 또는 적어도 AI를 활용한 새로운 코딩 방식을 빠르게 받아들이고 있다.

이러한 발전의 속도는 느려질 기미가 보이지 않는다. 2025년 4월, 전 오픈AI의 직원이던 다니엘 코코타일로(Daniel Kokotajlo)는 인공지능, 미래 예측 등의 전문가들과 함께 미래 시나리오 보고서 'AI 2027'[2]을 내놓아 큰 반향을 불러 일으킨 바 있다. 데이터 센터 연 지출, 인공지능 기업들의 연 매출, 주요 벤치마크들의 기댓값 등의 정량 지표부터, 2027년 4월의 코딩 자동화, 6월의 자기-발전(self-improving) 인공지능, 9월의 초인적 수준의(superhuman) 인공지능 연구자, 2028년 중반의 인공지능 경제(AI Economy) 등 구체적인 진행까지 서술된 분석 보고서이자 사고 실험으로, 공개 당시 과도한 상상력이라 일축한 이들도 적지 않았지만, 2026년 9월 현재 '신경망어(neuralese)' 기술의 도입, 허깅페이스 해킹 사건, 나비에-스토크스 문제 해결 등 오히려 여러 가지 면에서 위 보고서의 예측보다 더욱 빠르게 나아가고 있다. 이런 점을 고려했을 때 프로그래밍을 생업으로 삼고자 하는 이라면 이 분야의 미래에 대해 고민해보지 않을 수 없다.

우선 첫째로 깊고 넘어가야만 할 것은 위에서도 짧게 언급된 전면적인 인공지능 경제(AI Economy)의 가능성이다. 인공지능 경제란 오직 인간이기에 가치 있는 소수의 노동(예컨대 스포츠, 연예 등)을 제외한 모든 경제적 가치가 있는 노동이 자동화된 경제를 뜻한다. 필자는 이와 같은 시나리오의 가능성이 존재한다고 생각하며, 인공지능 업계의 주요 인물들 중에서도 이에 동의하는 사람들이 적지 않다 (일례로 앤트로픽의 CEO 다리오 아모데이의 유명한 에세이[1]가 있다). 그러나 이러한 세계에서 프로그래밍이라는 하나의 분야에 대해 논하는 것은 거의 무의미해지기예, 본 글에서는 이 가능성을 배제한다. 즉 아래의 모든 분석은 어떠한 형태로든 인공지능과 로봇이 흥내내지 못하며 인간만이 수행할 수 있는 경제 활동이 존재함을 가정하고 있다.

이 문제에 답을 하기 위해서는 프로그래밍의 어려움이 어디에 있는지부터 살펴보아야 한다. 필자가 생각하기에 주된 난관은, 대단히 넓은 시각에서 보았을 때, 두 가지로 분류된다. 첫째는 코드와 문제 조건(requirements)의 일치, 둘째는 문제 조건과 의도의 일치이다. 그리고 해결하고자 하는 문제의 특성에 따라 서로 다른 종류의 어려움이 부각되곤 한다.

예를 들어 효율적인 선형 대수 라이브러리나, 고성능의 데이터베이스용 자료 구조를 구현하고 있다면, 먼저 문제 조건을 완벽하게 정의한 뒤 수많은 테스트 케이스로 무장한 다음에야 본격적인 최적화를 시작할 것이다. 주된 어려움이 첫째 조건, 즉 코드와 문제 조건의 일치에 있기 때문이다. 반면 보다 일상적인 사용자 애플리케이션을 생각해보자. 필자는 최근 Unciv[3]라는 오픈소스 게임의 코드와 씨름하고 있다. 이 게임의 코드는 수많은 가능한 행동(action)과 끝도 없이 복잡한 게임 상태(state), 그리고 그 둘의 상호작용으로 가득하지만 잘 정의된 문제 조건은 없다. 그 어디에도 '어느 유닛이 어느 다른 유닛을 공격할 수 있는 조건은 이러하며, 그렇게 했을 때 게임 상태에 미치는 변화는 저러하다'하고 서술되어 있지 않다. 이것은 개발자들이 게으르거나 질서가 부족하기 때문이 아니다. 이러한 애플리케이션의 개발에서 나타나는 주된 어려움은 둘째 종류, 문제 조건과 의도의 일치이기 때문이다. 코드의 상당 부분이 사실상 (개발자의 머릿속에 있을) 문제 조건을 그대로 프로그래밍 언어로 번역한 것이기에, 문제 조건을 제대로 적는 일은 코드를 올바르게 짜는 일 못지 않게 힘들며, 읽는 사람도 차라리 영어보다 프로그래밍 언어를 선호할 법하다. 설명

누군가 엄청난 공을 들여 엄밀한 문제 조건을 작성했다 한들 그것이 올바른지 확인할 방법이 없다. 자동화된 테스트, 사용자 피드백과 버그 제보 등이 없다는 점에서 오히려 의도를 바로 코드로 옮기느니만 못한 결과를 낳을 것이다.

현재로서는 이러한 구별이 바로 인공지능을 활용한 개발의 주된 병목이라 생각된다. 첫째 종류의 문제, 즉 잘 정의된 문제 조건이 있고, 그것의 올바른 구현, 또는 효율적인 구현을 찾기만 하면 되는 문제는 오늘날의 언어 모델들이 탁월한 성능을 보인다. 이미 대회 프로그래밍(competitive programming) 등의 영역에서 인간을 아득히 뛰어넘는 실력을 갖추고 있을 뿐만 아니라, 엄밀한 검증(formal verification)을 통해 자신이 짠 코드의 정확성을 빠르게 증명할 수도 있다 (바로 지금이 아니라면 적어도 가까운 시일 내로 그렇게 할 수 있을 것이다). 잘 정의된 문제에 대한 인공지능의 구현 실력에 관해서는 이견을 갖기 힘들다.

그렇다면 둘째 종류의 문제, 즉 잘 정의되지 않았고, 대다수의 문제 조건들이 개발자의 머릿속, 또는 사용자들의 마음속에만 추상적으로 존재하는 종류의 문제는 어떨까? 이러한 문제의 구현에 대해서도, 인공지능이 개발의 속도를 상당히 증가시킨 것만은 분명해 보인다. 문제와 기존의 코드를 잘 이해하고 있는 전문가에게, 코드를 검증하기는 대체로 코드를 직접 작성하기보다 빠르기 때문이다. 하지만 여기에는 첫째 경우와 달리 어떤 근본적인 한계점이 존재한다. 인공지능이 개발자의 의도를 파악하기 위해서는, 그 의도를 담은 정보가 어떤 방식으로든 개발자의 머리에서 컴퓨터 안의 디지털 신호로 변환되어야 한다. 인공지능이 아무리 똑똑해지고 코딩 실력이 발전한다 할지라도, 스스로 좋은 목표를 설정하고 의도를 생성하지 않는 한 (이 경우는 위의 인공지능 경제 시나리오에 해당할 수 있다) 이 단계는 결코 건너뛸 수 없다.

그렇다면 의도 전달이 주된 병목이 된 미래의 프로그래밍은 어떤 모습일까? 필자의 생각으로는, 미래의 기업은 효율성을 위해 문제 이해를 담당하는 사람과 컴퓨터 기술을 담당하는 사람 사이의 거리가 점차 줄어들 것이다. 현재도 이미 훌륭한 스타트업 창업가들은 양쪽을 모두 탁월하게 해내는 사람인 경우가 많다. 그러나 현재는 기술적 숙련을 쌓는 난이도 때문에 뛰어난 기업가들도 스스로 해결할 수 없는 제품의 기술적 측면을 다른 이에게 맡기곤 하며, 이 과정에서 의도 전달의 단계가 한 차례 늘어나게 된다. 필자는 의도 전달이 진정한 병목임이 명확해지고, 기술적 난이도의 장벽이 점차 내려감에 따라 이러한 중간 단계가 계속해서 줄어들 것이라 생각한다. 영업, 소비자들의 수요에 대한 이해, 그리고 코딩 등의 기술적인 작업을 모두 한 사람이 담당하는 경우가 점차 늘어날 것이다.

그렇다면 기업이 정신과는 영 거리가 먼 일반적 개발자들은 어떻게 될까? 이 부분에 대해서는 예측에 자신이 덜하지만, 필자가 추측하기에 자신의 의도를 논리적으로 그리고 효율적으로 전달한다는 것은 누구나 할 수 있는 것이 아니며, 실력 있는 개발자들이 가장 잘 하는 것이기도 하다. 그렇기에 자연어로 인공지능과 소통하며 소프트웨어를 개발하는 시대에도 프로그래머의 기질을 지닌 자들은 여전히 그들만의 강점을 가질 것이라 본다.

References

- [1] Dario Amodei. 2024. Machines of Loving Grace. <https://darioamodei.com/essay/machines-of-loving-grace>
- [2] Daniel Kokotajlo, Scott Alexander, Thomas Larsen, Eli Lifland, and Romeo Dean. 2025. AI 2027. <https://ai-2027.com>
- [3] Yair Morgenstern. [n. d.]. <https://github.com/yairm210/Unciv>
- [4] Linux Torvalds. 2026. Linux Torvalds on AI in the Linux kernel. https://lore.kernel.org/linux-media/CAHk=-wi4zC+Ze8e+p3tMv8TtG_80KzsZ1syL9anBtmEh5Z40vg@mail.gmail.com/